

Construyendo paso a paso

Práctica: Gestión de Personas

Importante: en esta guía se muestran **partes del código** y la **arquitectura**.

No se incluyen las expresiones regulares (RegEx). En su lugar se dejan **marcadores** para que el docente las proporcione o el alumnado las implemente.

0) Qué vas a construir

Una app **por consola** que permita:

- Registrar **Alumno** y **Profesor**
- Listar personas
- Buscar por **ID** y por **documento**
- (Nivel 2) Evitar duplicados con **Set**
- (Nivel 2) Buscar por prefijo **con recursividad**
- Preparar la salida para la siguiente práctica con **CSV**

Ejemplo de línea CSV objetivo (en un futuro no muy lejano):

```
ALUMNO;1;Ana López;12345678A;ana@mail.com;2005-03-10;2026-02-26;1DAW
```

1) Arquitectura recomendada

Estructura de paquetes:

```
src/  
  com.docencia.app/  
    Main.java  
  com.docencia.model/  
    Persona.java  
    Alumno.java  
    Profesor.java  
  com.docencia.service/  
    CentroEducativo.java  
  com.docencia.util/  
    Validaciones.java
```

Idea clave:

- **model** = datos (clases)
- **service** = lógica de gestión (colecciones, búsquedas)
- **util** = validaciones reutilizables

- **app** = interfaz por consola (menú)

2) Paso 1 — Crear la clase abstracta Persona

2.1 Contrato mínimo

- **id** y **fechaRegistro** **inmutables** (final).
- Validar datos en constructor y setters.
- Calcular edad con **Period**.

2.2 Esqueleto de código

```
package com.docencia.model;

import java.time.LocalDate;
import java.time.Period;

public abstract class Persona {
    private final int id;
    private String nombre;
    private String documento;
    private String email;
    private LocalDate fechaNacimiento;
    private final LocalDate fechaRegistro;

    public Persona(int id, String nombre, String documento, String email,
                  LocalDate fechaNacimiento, LocalDate fechaRegistro) {

        if (id <= 0) throw new IllegalArgumentException("ID inválido");

        this.id = id;
        setNombre(nombre);
        setDocumento(documento);
        setEmail(email);
        setFechaNacimiento(fechaNacimiento);

        if (fechaRegistro == null ||
            fechaRegistro.isAfter(LocalDate.now())) {
            throw new IllegalArgumentException("Fecha de registro
            inválida");
        }
        this.fechaRegistro = fechaRegistro;
    }

    public final int getId() { return id; }
    public final LocalDate getFechaRegistro() { return fechaRegistro; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().length() < 2) {
            throw new IllegalArgumentException("Nombre inválido");
        }
    }
}
```

```

    }
    this.nombre = nombre.trim();
}

public String getDocumento() { return documento; }
public void setDocumento(String documento) {
    // La validacion la puedes hacer aqui, pero tienes que hacerte la
    // pregunta si tiene sentido dado que vas a hacer busquedas previsas en las
    // listas de Set(HahSet) y la List (ArrayList) es el cajon de sastre

    this.documento = normalizado;
}

public String getEmail() { return email; }
public void setEmail(String email) {

    // igual que en el caso anterior. Debes de plantearte como no
    // duplicar código ni responsabilidades

}

public LocalDate getFechaNacimiento() { return fechaNacimiento; }
public void setFechaNacimiento(LocalDate fechaNacimiento) {
    if (fechaNacimiento == null ||
        !fechaNacimiento.isBefore(LocalDate.now())) {
        throw new IllegalArgumentException("Fecha nacimiento
        inválida");
    }
    this.fechaNacimiento = fechaNacimiento;
}

public int getEdad() {
    // piensa como calcular la edad a partir de la fecha de nacimiento.
    // Un solución es con Period.between
    return 0;
}

public abstract String getTipo();

@Override
public String toString() {
    // Genera una representación de la persona
    return "";
}
}

```

Cuando finalices esta parte debes de tener: La clase `persona` finalizada y lista para utilizar.

3) Paso 2 — Crear Alumno y Profesor (herencia)

3.1 Alumno

```
package com.docencia.model;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.Set;

public class Alumno extends Persona {
    // Los alumnos pertenecen a un curso y tienen un conjunto de módulos en
    // dicho curso. Por ejemplo curso = 1DAM, y módulos de programación y ets.
    private String curso;
    private Set<String> modulos;

    public Alumno(int id, String nombre, String documento, String email,
        LocalDate fechaNacimiento, LocalDate fechaRegistro,
        String curso) {
        super(id, nombre, documento, email, fechaNacimiento,
            fechaRegistro);
        // ¿Qué debes de inicializar aquí?
    }

    @Override
    public String getTipo() { return "ALUMNO"; }

    public String getCurso() { return curso; }
    public void setCurso(String curso) {
        if (curso == null || curso.trim().length() < 3) {
            throw new IllegalArgumentException("Curso inválido");
        }
        this.curso = curso.trim();
    }

    public boolean addModulo(String modulo) {
        //Razona como se añade un módulo al conjunto de módulos y si pueden
        //existir módulos duplicados.
        return false;
    }

    public Set<String> getModulos() { return modulos; }

    @Override
    public String toString() {
        // Completa el toString para que muestre el tipo, el curso y los
        // módulos en los que esta matriculado el alumno.
        return "";
    }
}
```

Cuando finalices esta parte debes de tener: La clase `alumno` finalizada y lista para utilizar.

3.2 Profesor

```
package com.docencia.model;

import java.time.LocalDate;

public class Profesor extends Persona {
    private String departamento;

    public Profesor(int id, String nombre, String documento, String email,
                    LocalDate fechaNacimiento, LocalDate fechaRegistro,
                    String departamento) {
        super(id, nombre, documento, email, fechaNacimiento,
        fechaRegistro);
        //Razono como debes de inicializar el departamento
    }

    @Override
    public String getTipo() {
        // ¿Qué se debe de devolver aquí?
        return "";
    }

    public String getDepartamento() { return departamento; }
    public void setDepartamento(String departamento) {
        // Recuerda que verificaciones se deben de realizar en este set
    }

    @Override
    public String toString() {
        // Debes de retornar el tipo, y el departamento al que pertenece el
        profesor
        return "";
    }
}
```

4) Paso 3 — Crear Validaciones (sin regex aquí)

La clase existe para centralizar validación. Aquí solo dejamos el contrato:

```
package com.docencia.util;

public class Validaciones {

    private Validaciones() {}

    public static boolean emailValido(String email) {
        // TODO: implementar con RegEx
        return true;
    }
}
```

```
    public static boolean documentoValido(String documento) {  
        // TODO: implementar con RegEx  
        return true;  
    }  
}
```

Cuando finalices esta parte debes de tener: La clase `Validaciones` finalizada y lista para utilizar por otras clases.

5) Paso 4 — CentroEducativo (ArrayList + Set)

```
package com.docencia.service;  
  
import com.docencia.model.*;  
  
import java.util.ArrayList;  
import java.util.HashSet;  
import java.util.List;  
import java.util.Set;  
import com.docencia.util.Validaciones;  
  
public class CentroEducativo {  
    // Aqui esta el grueso del ejercicio y el razonamiento al que quiero  
    // que llegues  
    private final List<Persona> personas;  
    private final Set<String> documentosRegistrados;  
    private final Set<String> emailsRegistrados;  
  
    public CentroEducativo() {  
        // Que debes de inicializar aquí  
    }  
  
    public void registrarPersona(Persona persona) {  
        // Paso 1: Programación defensiva  
        // Paso 2: Evita elementos duplicados por el id. Con lo cual debes  
        // de buscar dentro de lista  
        // Paso 3: Evita que se pueda introducir un email con un formato o  
        // documento incorrecto  
        // Paso 4: Evita elementos duplicados por documento y email en los  
        // Set  
        // Paso 5: Incluye el documento dentro del set de documentos  
        // Paso 6: Incluye el email en el set de emails  
        // Paso 7: Incluye la persona dentro de la lista de personas  
  
    public List<Persona> listarPersonas() {  
        // Retorna la lista de personas  
        return null;  
    }  
}
```

```
public Persona buscarPorId(int id) {
    // Busca una persona por su id
    // Se inteligente y reutiliza esta funcion
    return null;
}

public Persona buscarPorDocumento(String documento) {
    // Busca una persona por su id
    // Se inteligente y reutiliza esta funcion
    return null;
}

public List<Alumno> listarAlumnos() {
    // A través de que parámetro te permite saber si una persona es de
    tipo alumno. Filtra a través de ese parámetro
    return null;
}

public List<Profesor> listarProfesores() {
    // A través de que parámetro te permite saber si una persona es de
    tipo profesor. Filtra a través de ese parámetro
    return null;
}

public List<Persona> buscarPorPrefijo(String prefijo) {
    // Función recursiva. Lee el README inicia para saber como
    implementar
    return null;
}
}
```

Cuando finalices esta parte debes de tener: Debes de poder registrar, listar y buscar (Alumnos/Profesores).

6) Paso 5 — Main (Vamos a crear un menú por consola)

6.1 Ejemplo de menú

```
===== CENTRO EDUCATIVO =====
1) Registrar alumno
2) Registrar profesor
3) Listar personas
4) Buscar por ID
5) Buscar por documento
6) Buscar por prefijo (recursivo)
0) Salir
Opción:
```

6.2 La parte que no sabes como construir: el `Main.java`

```
package com.docencia.app;

import com.docencia.model.Alumno;
import com.docencia.model.Persona;
import com.docencia.model.Profesor;
import com.docencia.service.CentroEducativo;

import java.time.LocalDate;
import java.util.List;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CentroEducativo centro = new CentroEducativo();

        int op;
        do {
            imprimirMenu();
            System.out.print("Opción: ");

            // Leer opción (control básico)
            String linea = sc.nextLine();
            try {
                op = Integer.parseInt(linea.trim());
            } catch (NumberFormatException e) {
                System.out.println("Opción inválida. Introduce un
número.");
                op = -1;
            }

            try {
                switch (op) {
                    case 1 -> registrarAlumno(sc, centro);
                    case 2 -> registrarProfesor(sc, centro);
                    case 3 -> listarPersonas(centro);
                    case 4 -> buscarPorId(sc, centro);
                    case 5 -> buscarPorDocumento(sc, centro);
                    case 0 -> System.out.println("Saliendo...");
                    default -> System.out.println("Opción no reconocida.");
                }
            } catch (Exception e) {
                System.out.println("ERROR: " + e.getMessage());
            }

        } while (op != 0);

        sc.close();
    }
}
```



```
private static void imprimirMenu() {
    System.out.println("\n===== CENTRO EDUCATIVO =====");
    System.out.println("1) Registrar alumno");
    System.out.println("2) Registrar profesor");
    System.out.println("3) Listar personas");
    System.out.println("4) Buscar por ID");
    System.out.println("5) Buscar por documento");
    System.out.println("0) Salir");
}

// === Registro (siguiendo tu estilo: nextLine + parse) ===

private static void registrarAlumno(Scanner sc, CentroEducativo centro)
{
    System.out.println("\n--- Registro Alumno ---");
    System.out.print("ID: ");
    int id = Integer.parseInt(sc.nextLine().trim());

    System.out.print("Nombre: ");
    String nombre = sc.nextLine();

    System.out.print("Documento: ");
    String documento = sc.nextLine();

    System.out.print("Email: ");
    String email = sc.nextLine();

    System.out.print("Fecha nacimiento (YYYY-MM-DD): ");
    LocalDate fechaNacimiento = LocalDate.parse(sc.nextLine().trim());

    LocalDate fechaRegistro = LocalDate.now();

    System.out.print("Curso: ");
    String curso = sc.nextLine();

    Alumno alumno = new Alumno(id, nombre, documento, email,
    fechaNacimiento, fechaRegistro, curso);
    centro.registrarPersona(alumno);

    System.out.println("Alumno registrado.");
}

private static void registrarProfesor(Scanner sc, CentroEducativo
centro) {
    System.out.println("\n--- Registro Profesor ---");
    System.out.print("ID: ");
    int id = Integer.parseInt(sc.nextLine().trim());

    System.out.print("Nombre: ");
    String nombre = sc.nextLine();

    System.out.print("Documento: ");
    String documento = sc.nextLine();
}
```

```
        System.out.print("Email: ");
        String email = sc.nextLine();

        System.out.print("Fecha nacimiento (YYYY-MM-DD): ");
        LocalDate fechaNacimiento = LocalDate.parse(sc.nextLine().trim());

        LocalDate fechaRegistro = LocalDate.now();

        System.out.print("Departamento: ");
        String departamento = sc.nextLine();

        Profesor profesor = new Profesor(id, nombre, documento, email,
        fechaNacimiento, fechaRegistro, departamento);
        centro.registrarPersona(profesor);

        System.out.println(" Profesor registrado.");
    }

    // === Otras opciones del menú ===

    private static void listarPersonas(CentroEducativo centro) {
        System.out.println("\n--- Listado de personas ---");
        List<Persona> personas = centro.listarPersonas();
        if (personas.isEmpty()) {
            System.out.println("(vacío)");
            return;
        }
        for (Persona p : personas) {
            System.out.println(p);
        }
    }

    private static void buscarPorId(Scanner sc, CentroEducativo centro) {
        System.out.println("\n--- Buscar por ID ---");
        System.out.print("ID: ");
        int id = Integer.parseInt(sc.nextLine().trim());
        Persona p = centro.buscarPorId(id);
        System.out.println(p == null ? "No encontrado" : p.toString());
    }

    private static void buscarPorDocumento(Scanner sc, CentroEducativo
    centro) {
        System.out.println("\n--- Buscar por documento ---");
        System.out.print("Documento: ");
        String documento = sc.nextLine();
        Persona p = centro.buscarPorDocumento(documento);
        System.out.println(p == null ? "No encontrado" : p.toString());
    }
}
```

***Checkpoint:** *el programa funciona y controla errores con `try/catch`.

9) ¿Qué es lo que debes de controlar?

- ☐ Herencia correcta (**Persona** abstracta + **Alumno** + **Profesor**)
- ☐ **CentroEducativo** con **ArrayList**, **Set** para evitar duplicados
- ☐ Recursividad por prefijo
- ☐ Menú funcional por consola
- ☐ **No utilizar la IA porque el ejercicio se da por inválido.**